Sesion 18 Slides

QGAN

Purpose. Introduce QGANs: choices (quantum/classical generator/discriminator), losses (BCE/Wasserstein), training patterns, stabilization strategies and practical lab choices for NISQ (small 1D mixtures, BAS).

Key design choices

- Which components are quantum?
  - QG-CD: quantum generator, classical discriminator (most practical on NISQ).
  - CG-QD: classical generator, quantum discriminator (researchy).
  - QG-QD: fully quantum (educational; small scale).
- Data model: classical bitstring targets (e.g., 4-bin or BAS patterns) are easiest.

Generator

- PQC producing a distribution over bitstrings by measurement.
- Typical ansatz: `TwoLocal` (RY/RZ + entanglers), 1–2 reps. Keep depth minimal.
- Noise / re-upload: you can add classical noise as inputs or quantum noise qubits

Discriminator

- Classical MLP on bitstrings is the simplest and stabilizes training.
- Quantum discriminator is doable (PQC + expectation head), but requires careful design and readout mitigation.

Losses & training

- BCE GAN: standard but can be unstable.
- Non-saturating generator loss: `-E[log D(G(z))]`.
- WGAN (critic) often more stable; requires careful Lipschitz constraint (gradient penalty or spectral norm).
- Gradients: parameter-shift works for expectation-based losses; for sampling-based losses (we compare histograms) gradient-free optimizers (SPSA, COBYLA) are simple and robust on NISQ.

Stabilization

- Label smoothing, instance noise, entropy regularizer for the generator, small learning rates.
- Barren plateau mitigation: shallow ansatz, problem-informed circuits, identity init, layerwise growth.

Evaluation

- Histogram distances (KL, JS), mode coverage (how many target modes captured), visual sample histograms.
- Always report shot budget, circuit depth, and sampling wall-clock.

Session 19 - Autoencoders

Purpose. Teach QAEs practically: encode → compress (latent) + trash qubits driven to $|0\ldots0\rangle$, train using a trash-state loss on small families of states, evaluate compression and (optionally) fidelity.

A quantum autoencoder (QAE) learns a parameterized encoder $U_E(\theta)$ that maps n input qubits into an m-qubit latent register plus $t = n - m$ "trash" qubits. The training objective (NISQ-friendly) is to **maximize the probability that trash qubits are measured as** $|0\rangle$ after encoding. Optionally train/measure decoder or compute reconstruction fidelity via swap test. Use shallow TwoLocal ansätze and SPSA for robust optimization on noisy hardware.

Key components & choices

- **Ansatz (encoder)**: TwoLocal (RY/RZ + CX), 1–2 reps, identity-ish init (small angles) to reduce barren-plateau risk.
- **Losses**:
  - *Trash-state loss*: $L = 1 - P(\text{trash} = 0)$. Cheap: measure only trash qubits.
  - *Fidelity loss*: requires decoder + swap-test between $|\psi\rangle$ and reconstructed state (higher cost).
  - *Observable loss*: compare a small set of observables if target has known characteristics.
- **Optimizers**: SPSA or Nelder–Mead (shot-noise robust). Parameter-shift via `Estimator` is possible on simulators/low-noise devices.
- **Data**: quantum states (Bell pairs, product states) or classical data encoded into states (angle/amplitude).
- **Evaluation**: trash probability curve, swap-test fidelity on held-out set, downstream task (use latent for classification).

Practical tips

- Start with the *trash-state* objective and SPSA.
- Initialize encoder parameters near zero (identity) to help trainability.
- Use small batches (2–8) of training states per SPSA gradient step to average noise.
- Cache transpiled circuits and bind parameters to avoid re-transpilation overhead.
- Apply readout error mitigation for small trash probabilities if using hardware.
- Report qubit count, depth, shot budget, and wall-clock for reproducibility.

```
# ------------------------------
# 1) Problem setup
# ------------------------------
n = 4          # total qubits (input)
m = 2          # latent qubits to keep
t = n - m      # trash qubits to drive to |0>
latent_qubits = [0, 1]   # indices we will keep
trash_qubits  = [2, 3]   # indices we want to be |0>

```

```python
def prep_input(idx: int) -> QuantumCircuit:
    qc = QuantumCircuit(n)
    # three simple patterns, cycle through them
    if idx % 3 == 0:
        # product-like
        qc.h(0); qc.ry(0.4, 1); qc.h(2)
    elif idx % 3 == 1:
        # small entanglement on (0,1)
        qc.h(0); qc.cx(0, 1)
        qc.ry(0.2, 2); qc.x(3)
    else:
        # entangle (1,2)
        qc.h(1); qc.cx(1, 2); qc.ry(0.3, 3)
    return qc
```

```python
# ----------------------------
# 2) Encoder ansatz (TwoLocal)
# ----------------------------
encoder = TwoLocal(
    num_qubits=n,
    rotation_blocks=['ry', 'rz'],
    entanglement_blocks='cx',
    entanglement='linear',
    reps=1,
    insert_barriers=False
)
num_params = encoder.num_parameters
print(f"Encoder with {n} qubits, params: {num_params}")

# start near identity (small angles)
theta0 = np.zeros(num_params)
```

```python
def projector_trash00(n_qubits, trash):
    from itertools import product
    terms = []
    coeffs = []
    # enumerate all combinations of I/Z on trash qubits (t qubits => 2^t terms)
    for bits in product([0, 1], repeat=len(trash)):
        label = ['I'] * n_qubits
        for b, q in zip(bits, trash):
            label[q] = 'Z' if b == 1 else 'I'
        # Qiskit SparsePauliOp expects string in little-endian order -> reverse the label
        pauli_str = ''.join(label[::-1])
        coeffs.append(1.0 / (2**len(trash)))
        terms.append(pauli_str)
    return SparsePauliOp.from_list(list(zip(terms, coeffs)))

P_trash00 = projector_trash00(n, trash_qubits)
```

```python
estimator = Estimator()

def trash_zero_prob(theta, batch_ids):
    """Return average probability P(trash=00) over the batch of input indices."""
    bound_encoder = encoder.assign_parameters(theta)
    circuits = []
    for idx in batch_ids:
        qc = QuantumCircuit(n)
        qc.compose(prep_input(idx), inplace=True)
        qc.compose(bound_encoder, inplace=True)
        circuits.append(qc)
    # call estimator: circuits list and same observable for each
    job = estimator.run(circuits, [P_trash00] * len(circuits))
    res = job.result()
    # res.values is array of expectation values
    vals = np.array(res.values, dtype=float)
    return float(vals.mean())
```

```python
def spsa_train(theta_init, steps=120, a0=0.12, c0=0.08, batch_size=4, seed=0, shots=None):
    """
    SPSA loop that maximizes P(trash=00) -> equivalently minimizes loss = 1 - P.
    Uses estimator internally (Estimator provides expectations; shots parameter optional).
    """
    rng = np.random.default_rng(seed)
    theta = theta_init.copy()
    best_theta = theta.copy()
    best_p = 0.0
    history = []
    for k in range(1, steps + 1):
        ak = a0 / (k ** 0.602)
        ck = c0 / (k ** 0.101)
        # batch of random training states
        batch_ids = rng.integers(0, 12, size=batch_size)
        # Rademacher perturbation
        delta = rng.choice([-1.0, 1.0], size=theta.size)
        thetap = theta + ck * delta
        thetam = theta - ck * delta
```

```
        # evaluate probabilities (averaged over batch)
        p_plus = trash_zero_prob(thetap, batch_ids)
        p_minus = trash_zero_prob(thetam, batch_ids)
        # loss = 1 - p, gradient estimate (for minimising loss)
        ghat = ( (1-p_plus) - (1-p_minus) ) / (2.0 * ck * delta)  # shape (num_params,)
        # update (we want to maximize p -> minimize loss, so step negative gradient)
        theta = theta - ak * ghat
        # evaluate current probability on same batch for reporting
        p_now = trash_zero_prob(theta, batch_ids)
        history.append((k, p_now))
        if p_now > best_p:
            best_p = p_now
            best_theta = theta.copy()
        if k % 10 == 0 or k == 1:
            print(f"SPSA step {k:3d} | P_trash00 ≈ {p_now:.4f} | best ≈ {best_p:.4f}")
    return best_theta, best_p, history
```

```
print("\nStarting SPSA training (trash-state objective). This uses Estimator to compute expectati
start = time.time()
best_theta, best_p, hist = spsa_train(theta0, steps=80, a0=0.12, c0=0.08, batch_size=4, seed=1)
end = time.time()
print(f"\nTraining finished in {end-start:.1f}s. Best P_trash00 ≈ {best_p:.4f}")

# Show final average probability over a held-out validation set
val_ids = list(range(12, 18))  # some other indices for validation
p_val = trash_zero_prob(best_theta, val_ids)
print(f"Validation P_trash00 (avg over {len(val_ids)} states) ≈ {p_val:.4f}")

```

```python
def swap_test_fidelity(theta, idx, shots=2048):
    # Prepare circuits for swap test: ancilla + 2 * n registers
    from qiskit import QuantumCircuit as QC
    anc = 1
    qc_st = QC(1 + 2 * n)
    # prepare original |psi> on reg A (qubits 1..n)
    qcA = QuantumCircuit(n)
    qcA.compose(prep_input(idx), inplace=True)
    qc_st.compose(qcA, qubits=range(1, 1 + n), inplace=True)
    # prepare reconstructed |psi_hat> = U_E^\dagger U_E |psi> -> effectively just apply encoder t
    qcB = QuantumCircuit(n)
    qcB.compose(prep_input(idx), inplace=True)
    qcB.compose(encoder.bind_parameters(theta), inplace=True)
    qcB.compose(encoder.bind_parameters(theta).inverse(), inplace=True)  # decoder ~ encoder^dagg
    qc_st.compose(qcB, qubits=range(1 + n, 1 + 2 * n), inplace=True)
    # swap test
    qc_st.h(0)
    for q in range(n):
        qc_st.cswap(0, 1 + q, 1 + n + q)
```

```
    qc_st.h(0)
    sampler = Sampler()
    job = sampler.run([qc_st], shots=shots)
    res = job.result()
    qd = res.quasi_dists[0]
    # robust extraction of ancilla=0 prob
    try:
        pdict = qd.binary_probabilities()
    except Exception:
        pdict = qd.nearest_probability_distribution().binary_probabilities()
    p0 = 0.0
    for bits, p in pdict.items():
        key = bits.replace(" ", "")
        # ancilla is first bit; treat any ordering defensively
        if len(key) >= 1 and key[0] == '0':
            p0 += p
    fidelity = max(0.0, min(1.0, 2 * p0 - 1))
    return fidelity

# pick one validation state
fidelity_est = swap_test_fidelity(best_theta, idx=12, shots=2048)
print(f"Swap-test approximate fidelity on idx=12: F ≈ {fidelity_est:.4f}")

```

```
Starting SPSA training (trash-state objective). This uses Estimator to compute expectations.
SPSA step   1 | P_trash00 ≈ 0.7417 | best ≈ 0.7417
SPSA step  10 | P_trash00 ≈ 0.8616 | best ≈ 0.8616
SPSA step  20 | P_trash00 ≈ 0.6125 | best ≈ 0.8616
SPSA step  30 | P_trash00 ≈ 0.2539 | best ≈ 0.8616
SPSA step  40 | P_trash00 ≈ 0.3723 | best ≈ 0.8616
SPSA step  50 | P_trash00 ≈ 0.2502 | best ≈ 0.8716
SPSA step  60 | P_trash00 ≈ 0.2510 | best ≈ 0.8716
SPSA step  70 | P_trash00 ≈ 0.3713 | best ≈ 0.8716
SPSA step  80 | P_trash00 ≈ 0.6230 | best ≈ 0.8716

Training finished in 99.4s. Best P_trash00 ≈ 0.8716
```